

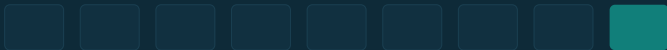


STREAM PROCESSING AVEC APACHE KAFKA

Anatomie distribuée de Kafka

Topics, partitions, offsets, brokers — et le mode KRaft

ESGI · 5^e année · Data Engineering · 2025–2026



Objectifs de la séance

- **Comprendre la décomposition topic → partitions**

La partition est l'unité réelle de stockage, d'ordre et de parallélisme.

- **Maîtriser le rôle des clés et des offsets**

Comment un message choisit sa partition, et ce qu'un offset identifie.

- **Visualiser un cluster : brokers, leaders, réplicas**

Et le mode KRaft qui gère les métadonnées sans ZooKeeper.

- **Manipuler un cluster à 3 brokers**

Créer un topic partitionné et observer sa répartition réelle.

Au programme



01

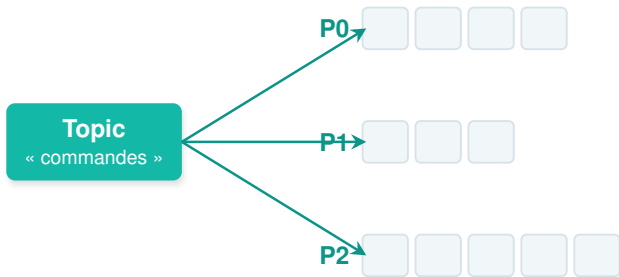
- PARTIE 1

Du topic aux partitions

Un topic est découpé en partitions

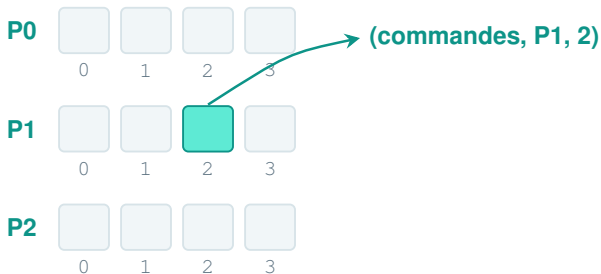
En S1 : un topic = un log. Mais un seul log = un seul fichier, un seul flux d'écriture.

La solution : découper le topic en partitions — chacune étant un log indépendant.



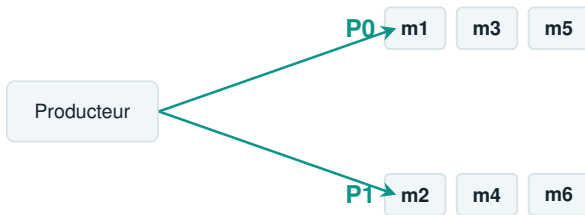
Chaque partition est un log ordonné, immuable et indépendant.

L'offset est propre à chaque partition



Le triplet (topic, partition, offset) adresse un message de façon unique. Les offsets ne sont pas comparables d'une partition à l'autre.

L'ordre est garanti par partition — pas globalement



*Dans P0 : $m1 < m3 < m5$. Mais rien ne garantit l'ordre de m3 par rapport à m4.
Besoin d'un ordre strict entre messages ? Ils doivent partager la même partition.*

Pourquoi partitionner ?

■ Débit

Écritures et lectures réparties sur plusieurs logs en parallèle.

■ Scalabilité

Les partitions se répartissent sur plusieurs brokers (Partie 3).

■ Parallélisme

Une partition = une unité de consommation parallèle (cf. consumer groups, S5).

La partition est l'unité de parallélisme de Kafka : c'est elle qu'on multiplie pour aller plus vite.

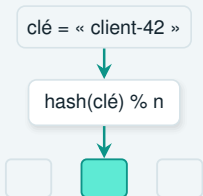
02

- PARTIE 2

Clés, partitionnement & offsets

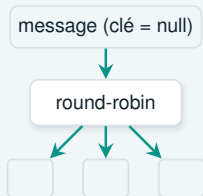
Comment un message choisit sa partition

■ Avec une clé



Même clé → toujours la même partition.

■ Sans clé

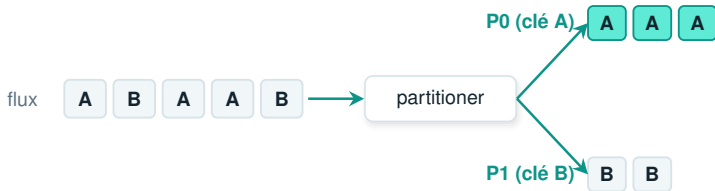


Réparti sur toutes les partitions.

Le partitionnement par défaut : hachage de la clé, ou répartition équilibrée si la clé est absente.

CONSÉQUENCE

La clé décide du regroupement et de l'ordre



Exemple : clé = identifiant client $\Rightarrow$ tous les événements d'un client tombent dans la même partition, donc strictement ordonnés.

Combien de partitions ?

■ Plus de partitions

Plus de parallélisme et de débit possible côté consommateurs.

■ Mais pas gratuit

Plus de fichiers ouverts, de métadonnées, d'élections de leader.

■ Repartitionner casse l'ordre

Changer n modifie le hachage : la clé peut changer de partition.

■ Règle pratique

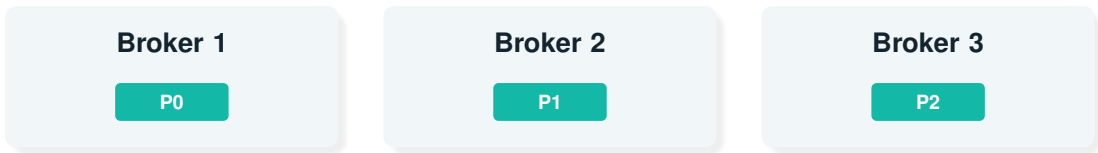
Prévoir un peu large dès le départ : augmenter est simple, réduire ne l'est pas.

03

- PARTIE 3

Brokers, cluster & KRaft

Les partitions sont réparties sur les brokers



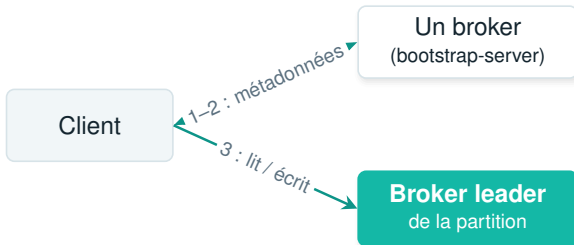
Un cluster = plusieurs brokers (serveurs). Les 3 partitions de « commandes » se répartissent sur les 3 brokers → charge et débit distribués.

Chaque partition : un leader + des réplicas



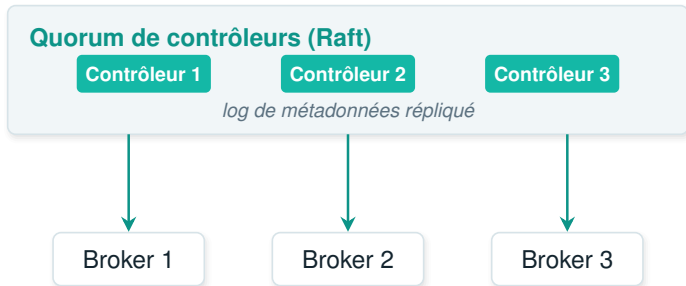
Avec un facteur de réplication de 3 : chaque partition a 1 **leader** (lectures/écritures) et 2 **réplicas** sur d'autres brokers. Si un broker tombe, un réplica prend le relais.
(durabilité, ISR, acks → détaillés en Séance 3.)

Comment un client trouve le bon broker



N'importe quel broker peut donner la carte du cluster ; ensuite, le client parle directement au leader de chaque partition.

KRaft : les métadonnées sans ZooKeeper



Depuis Kafka 4.x : plus de ZooKeeper. Topics, partitions, leaders et configs vivent dans un log géré par un quorum de contrôleurs. Un nœud joue le rôle `broker`, `controller`, ou les deux.

ZooKeeper → KRaft : ce que l'on gagne

■ Un seul système

Plus de ZooKeeper à déployer, sécuriser et superviser à part.

■ Passage à l'échelle

Des millions de partitions par cluster, là où ZooKeeper plafonnait.

■ Bascules plus rapides

Métadonnées en log : redémarrages et reprises accélérés.

■ Source unique

Cohérence via Raft : une seule source de vérité des métadonnées.

04

• PARTIE 4 · ATELIER

Un cluster à 3 brokers

Docker Compose : 3 brokers en KRaft

On étend le `compose` de la S1 : trois brokers, chacun avec un `NODE_ID` unique et le même quorum.

`docker-compose.yml` (extrait)

```
services:
  broker-1:
    image: apache/kafka:4.3.0
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_CONTROLLER_QUORUM_VOTERS:
        1@broker-1:9093,2@broker-2:9093,3@broker-3:9093
    # broker-2, broker-3 : meme config (NODE_ID 2, 3)
  kafka-ui:
    image: provectuslabs/kafka-ui
```

Créer et observer un topic partitionné

- 1 Lancer le cluster 3 brokers
- 2 Créer commandes (RF 3)
- 3 Lire le `--describe`
- 4 Répartition dans Kafka UI
- 5 Produire avec des clés
- 6 Même clé → même partition

bash

```
# 3 partitions, répliquées 3 fois
kafka-topics.sh --create --topic commandes \
  --partitions 3 --replication-factor 3 \
  --bootstrap-server localhost:9092
# leaders, replicas, ISR
kafka-topics.sh --describe --topic commandes \
  --bootstrap-server localhost:9092
# produire avec une cle (cle:valeur)
kafka-console-producer.sh \
  --topic commandes \
  --property parse.key=true \
  --property key.separator=: \
  --bootstrap-server localhost:9092
```

Décoder la sortie de `--describe`

commandes `--describe`

```
Topic: commandes Partitions: 3 RF: 3
Partition: 0 Leader: 1 Replicas: 1,2,3 Isr: 1,2,3
Partition: 1 Leader: 2 Replicas: 2,3,1 Isr: 2,3,1
Partition: 2 Leader: 3 Replicas: 3,1,2 Isr: 3,1,2
```

■ Leader

Sert les lectures et écritures de la partition.

■ Replicas

Les brokers qui en détiennent une copie.

■ Isr

Réplicas synchronisés, à jour (voir S3).

Récapitulatif

- ✓ Un topic est découpé en partitions ; chaque partition est un log ordonné avec ses propres offsets.
- ✓ L'ordre n'est garanti qu'au sein d'une partition.
- ✓ La clé décide de la partition (hachage) → regroupement et ordre par clé.
- ✓ Un cluster répartit les partitions sur des brokers ; chacune a un leader et des réplicas.
- ✓ KRaft remplace ZooKeeper : les métadonnées vivent dans un quorum de contrôleurs.



LA SUITE — SÉANCE 3

Réplication & durabilité

ISR, acks, rétention et compaction des logs — puis un TP sur la tolérance aux pannes : on arrête un broker et on observe le cluster réagir.